
MitiCon for Mitigating Model Contamination in Distributed ML for UAVaaS

— Presentation by Ryena Dhingra —

Reference

Kumar, Prince, Veera Manikantha Rayudu Tummala, and Arijit Roy. "MitiCon: Mitigating Model Contamination in Distributed Machine Learning for UAV-as-a-Service." IEEE Transactions on Vehicular Technology (2025): 1–11. Accepted for publication.

Distributed ML for UAVaaS

- In current UAVaaS platforms, machine learning is distributed across UAVs, rather than trained at a single central location
- Each UAV:
 - Collects data from its sensors
 - Trains an ML model locally
 - Transmits all data and model to a central server where a global model is built
- Available approaches for aggregating drone model:
 - FedAvg
 - FedProx
- Distributed ML for UAVaaS is needed to provide services like
 - Smart disaster management
 - Smart traffic monitoring

Problems with Current Approach

1. Model Contamination
2. High Communication Latency and Overhead

Model Contamination

- *The degradation of machine learning model performance caused by training of model with erroneous data, collected by UAVs*
- During data collection, a UAV may encounter:
 - Faulty sensors
 - Adverse weather
 - Malfunctions
- Leads to erroneous or corrupted data collection
- Locally trained model learns incorrect patterns
- Contaminated model is aggregated with other models at central server
- Leads to incorrect predictions at the global level

Model Contamination

- Why does it matter?
- The effectiveness of UAV based IoT applications depends on the accuracy and reliability of data collected by UAVs
- Training models with such data degrades decision-making performance, affecting end-user applications
- The stakes are even higher when many UAVs collaborate to serve multiple applications
- Conclusion: Including incorrect data undermines the accuracy and reliability of ML models, adversely affecting the decision-making ability of UAVaaS platform

High Communication Latency and Overhead

- Existing approaches transmit all data and models to a central server
- Hence, we observe
 - High communication latency
 - High communication overhead
- This limits responsiveness in time-sensitive applications

The Need for MitiCon

- An utmost requirement to design a scheme that
 - Ensures model contamination mitigation
 - Reduces the latency in data transmission
- How does MitiCon address these problems?

A Brief Introduction

- Miticon allows UAVs to transmit only their locally trained models and a small, representative subset of data– termed ValiData to the central server
- A scaled voting mechanism evaluates ValiData's trustworthiness
- The TrustLoss metric identifies contaminated models
- Further, a softmax-based aggregation strategy then combines UAV machine learning models, giving greater weight to validated models

Problem Description

- Applications often require extensive data collection, and the accuracy and timeliness of UAV-collected data are critical for their effectiveness
- A major challenge in UAVaaS platforms is the collection of erroneous data
- If undetected, such data contaminates machine learning models
- A reliable scheme is needed to identify and isolate models trained on inaccurate data
- Real-time detection is critical
- Core problem: to identify UAVs with contaminated models and mitigate their effects to ensure accurate and reliable performance in the UAVaaS platform

Mathematical Modelling of MitiCon

Describing the System

- Set of UAVs, $U = \{u_1, u_2, \dots, u_n\}$
- Each j^{th} UAV has a set of sensors, $S = \{s_1^j, s_2^j, \dots, s_m^j\}$
- UAVs are managed by a set of owners, $O = \{o_1, o_2, \dots, o_p\}$
- $F \subseteq S$ is the set of faulty sensors
- $V \subseteq S$ is the set of faulty UAVs (which contain at least 1 faulty sensor)
- UAV u_i senses data $D_{ui} = x_{i1}, x_{i2}, \dots, x_{ik}$ with k data points per UAV
- Total dataset $D = \bigcup_{i=1}^n D_{ui}$
- Erroneous data points, $D_f \subseteq D$

Describing the System

- Model contamination occurs when UAVs include D_f in their training
- This produces contaminated models, $M_f = m_{f1}^t, m_{f2}^t, \dots, m_{fy}^t$
- Contaminated models degrade UAVaaS performance by generating unreliable predictions and incorrect end-user decisions
- MitiCon aims to detect UAVs V contributing to D_f and mitigate model contamination by minimizing the influence of M_f during aggregation

Describing the System

- At application start, each UAV $u_i \in U$ receives a base model m_i from the central server
- During execution, UAV u_i senses data D_{ui}
- After collecting data, each UAV trains its base model on its local dataset
- Let m_i^t denote the model trained by UAV u_i on D_{ui}
- $M_i^t = T(m_i, D_{ui}) \quad \forall u_i \in U$
- $T(m_i, D_{ui})$ represents the training process that updates the base model m_i using D_{ui}
- The set of all trained models is represented as $M^t = \{m_1^t, m_2^t, \dots, m_n^t\}$

ValiData

- Sending the entire dataset D collected by each UAV to the central server leads to significant delay and communication overhead
- To deal with this we use *ValiData*
- From the total data set D_{u_i} of each UAV, a small portion called ValiData (d_{u_i}) is selected
- Each u_i sends its respective ValiData and model to the central server
- We carefully choose this subset of data to represent the UAV's overall sensing experience
- $$d_{u_i} = \arg \max_{\substack{d \subseteq D_{u_i} \\ |d| \leq k}} \left(I(M, A) - \lambda \cdot R(d) \right) \quad (1)$$

What does ValiData do?

- With the help of ValiData, communication overhead and processing delays will be minimal, and the data will be tested to identify the faulty UAV
- The ValiData from each UAV are evaluated by model of each UAV at the central server
- This evaluation process is called *CrossEval*
- CrossEval ensures that each ValiData is tested across all models, making the validation process robust
- The central server runs each model m_i^t on all ValiData subsets (d^s), ensuring that the (d^s) is cross validated for all models m^t

CrossEval

- CrossEval (C_{ij}) is defined as a method where the central server computes the loss (L_{ij}) for each ValiData $d_{ui} \in d_s$, when evaluated on each model $m_j^t \in M^t$
- $C_{ij} = f(d_{ui}, m_j^t), \quad \forall m_j^t \in M^t, d_{ui} \in d^s \quad (2)$
- Loss (L_{ij}) in MitiCon represents the error when the ValiData d_{uj} is evaluated on model m^t
- $L_{ij} = \text{Loss}(m_i^t, d_{uj}) = 1 - \text{Accuracy}(m_i^t, d_{uj})$

Voting

- Loss for each CrossEval is compared against a predefined threshold t_{loss} , a hyperparameter
- v_j is the vote assigned to the respective ValiData d_{uj} based on the calculated loss
- $$v_j = \begin{cases} +1, & \text{if } L_{ij} < t_{\text{loss}} \\ 0, & \text{if } L_{ij} \geq t_{\text{loss}} \end{cases} \quad (3)$$
- If L_{ij} for model m_i^t on data d_{uj} is less than t_{loss} , it is considered to have performed well on ValiData d_{uj} and a positive vote ($v_j = +1$) is assigned to the UAV ValiData

Vote Vector

- The voting mechanism ensures that all ValiData gets validated on all models
- Once CrossEval completes, a vote vector v_n is generated
- $$v_n = [v_{d_{u_1}} \quad v_{d_{u_2}} \quad \dots \quad v_{d_{u_j}} \quad \dots \quad v_{d_{u_n}}] \quad (4)$$
- In v_n , each element $v_{d_{uj}}$ represents the number of positive votes received to ValiData d_{uj}
- $v_{d_{u_j}} = \sum_i v_j$ where each v_j is an individual vote for d_{uj} , taking a value of either 0 or 1 based on model's performance
- MitiCon's voting is based purely on model performance

Min-Max Scaling the Votes

- Min-max scaling approach to normalize vote values, ensuring they are on a consistent scale
- Since votes represent different UAVs or ValiData, their ranges vary
- Without normalization, some votes may disproportionately influence the final result, especially if they have larger values
- Min-max scaling transforms votes into a uniform range, allowing fair comparison and ensuring no single vote dominates

- $$\hat{v}_{d_{u_j}} = \frac{v_{d_{u_j}} - \min(v_n)}{\max(v_n) - \min(v_n)} \quad (5)$$

Mini Dataset

- Models trained on erroneous data tend to perform poorly on reliable data
- To identify such models, we construct a mini dataset $\tilde{D}$
- Mini dataset $\tilde{D}$ contains only reliable data
- The purpose of this mini dataset is to ensure that only trustworthy data is used, minimizing the impact of erroneous data on the UaaS platform
- We construct the mini dataset by computing the dot product between scaled votes $\hat{v}_n$ and each ValiData d_{ui}
- This ensures that only the most reliable data, weighted by the scaled votes, is included while erroneous data is removed

- $$\tilde{D} = \sum_{i=1}^n \hat{v}_{d_{u_i}} \cdot d_{u_i} \quad (6)$$

TrustLoss

- We compute each model's (m_j^t) loss by evaluating it on the mini dataset $\tilde{D}$
- This is quantified using a metric called *TrustLoss*, which measures the performance of model m_j^t in M^t based on trustworthy and representative data
- *TrustLoss is defined as the process of calculating the loss for each model m_j^t based on the mini dataset $\tilde{D}$*
- $L_{\tilde{D}}(m_j^t) = \text{Loss}(m_j^t, \tilde{D})$ (7)

Inverse of TrustLoss

- After computing TrustLoss for each model, we calculate its inverse as

$$L_{\tilde{\mathcal{D}}}^i(m_j^t) = \frac{1}{L_{\tilde{\mathcal{D}}}(m_j^t)} \quad (8)$$

- Eq 8 assigns greater weight to models with lower TrustLoss, emphasizing their superior performance on the mini dataset
- Now models with smaller errors receive higher weights, while those with larger errors are down-weighted, minimizing their influence on model aggregation
- This ensures that the most accurate models contribute significantly to the UAVaaS platform, enhancing the robustness and accuracy

Applying Softmax

- *The TrustLoss metric effectively minimizes or completely removes the influence of contaminated models in the aggregation process by assigning smaller values to models trained on erroneous data in the UaaS platform*
- After computing the inverse of TrustLoss, we apply the softmax function to $L_{\mathcal{D}}^i(m_j^t)$
- We apply softmax for two reasons:
 - Inverse loss values can vary greatly → Using raw inverses would bias weight distribution and complicate comparisons between reliable and contaminated models
 - Inverse losses are not normalized → so their sum may not be 1; without softmax, weights would be uneven, making it hard to assess each model's contribution

Applying Softmax

- Softmax converts inverse losses into normalized weights that sum to 1, ensuring fair distribution
- By inverting TrustLoss and applying softmax, models with smaller TrustLoss receive higher weights, while those with larger TrustLoss receive lower weights, giving better-performing models more influence on the final aggregation
- Weight of i^{th} model is α_i

- $$\alpha_i = \text{Softmax} \left(L_{\mathcal{D}}^i(m_j^t) \right) \quad (13)$$

$$\text{Softmax}(z_i) = \frac{e^{z_i}}{\sum_{j=1}^n e^{z_j}} \quad (14)$$

AggrModel

- After applying softmax, we perform *AggrModel* and return the updated model to each UAV
- *AggrModel* refers to weighted model aggregation into a single unified model M' , where each $m_i^t \in M$ is weighted by its corresponding performance-based weight α_i
- $$M' = \sum_{i=1}^n \alpha_i \cdot m_i^t \quad (15)$$
- The aggregated model M' serves as the final unified model, ensuring contributions of contaminated models are minimized

AggrModel

- The updated model M' is then distributed back to each UAV, completing the collaborative learning process and preparing the system for the next iteration or application

MitiCon Algorithm

Algorithm 1 MitiCon

```
1: Input:  $U = \{u_1, u_2, \dots, u_n\}$ ,  $M = \{m_1, m_2, \dots, m_n\}$ ,  
    $D = \{D_{u_1}, D_{u_2}, \dots, D_{u_n}\}$ ,  $d_{u_i} \subseteq D_{u_i}$ ,  $t_{\text{loss}}$   
2: Output: Updated model  $M'$   
3: Initialization:  $u_i \leftarrow m_i$  from the server  
4: Each UAV  $u_i$  senses data  $D_{u_i}$  based on application  
5: Each  $u_i$  trains initial model  $m_i$  from sensed data  
6: for each UAV  $u_i \in U$  do  
7:   Select ValiData  $d_{u_i}$  using Eq. (1)  
8:   Sends its model  $m_i^t$  and ValiData  $d_{u_i}$  to the central  
   server  
9: end for  
10: Central server perform CrossEval ( $C_{ij}$ )  
11: for each ValiData  $d_{u_j} \in d$  do  
12:   for each model  $m_i^t \in M^t$  do  
13:     Compute loss  $L_{ij} = \text{Loss}(m_i, d_{u_j})$   
14:     if  $L_{ij} < t_{\text{loss}}$  then  
15:       Set vote  $v_{d_{u_j}} = +1$   
16:     end if  
17:   end for  
18: end for  
19: Perform vote normalization using Eq. (5) on votes  
20: Construct Mini dataset using Eq. (6)  
21: Calculate TrustLoss (Eq. (7)) and its inverse (Eq. (8))  
22: Apply softmax to Eq. (8) using Eq. (13) for  $\alpha$   
23: Perform AggrModel using Eq. (15)  
24: Return the updated model  $M'$  to UAVs
```





